

API SECURITY TOOLS AND TOOL OUTPUT REPORT

Enterprise Security Assessment Lab

Professional Revised Edition - Proper Screenshot Alignment and Sanitized Tool Outputs

Prepared By	Jagriti Banerjee
Module	Sub-Project 2 - API Security
Lab Scope	crAPI REST API, DVGA GraphQL, OAuth lab request analysis
Report Focus	Tools, commands, sanitized outputs, screenshot evidence, interpretation, remediation
Publication Format	GitHub-safe PDF with aligned figure blocks

This revised version places screenshots inside consistent evidence frames and separates technical explanation from figure pages. The result is cleaner, easier to read, and more professional for GitHub portfolio review.

Table of Contents

#	Section
1	Executive Summary
2	API Security Toolchain Overview
3	Evidence and Output Matrix
4	Docker and Lab Availability Tool Output
5	Postman API Workflow and Authentication Output
6	JWT Tooling and Token Analysis Output
7	Burp Suite Proxy and Repeater Output
8	Kiterunner and ffuf Output Interpretation
9	GraphQL Tool Output Review
10	OAuth Request Analysis Output
11	Sanitized Output Library
12	GitHub Redaction and Reporting Guidance
13	Appendix - Command Reference

1. Executive Summary

This report documents the API Security tools and tool outputs used in the Enterprise Security Assessment Lab. It explains not only which tools were used, but what their outputs prove, how each output should be interpreted, what risks the output relates to, and how the evidence should be safely published on GitHub.

The API module includes crAPI deployment, Postman request workflows, JWT authentication analysis, BOLA/IDOR authorization testing, data exposure review, rate-limit behaviour testing, mass-assignment control validation, Kiterunner endpoint discovery, DVGA GraphQL testing, and OAuth request analysis through Burp Suite.

The report avoids exaggeration. Rejected attacks such as JWT manipulation or mass assignment are documented as control validations. Confirmed weaknesses such as BOLA-style object access are explained with risk context and remediation.

Assessment Area	Primary Tools	Output Value
Deployment	Docker, Browser, curl	Confirms lab services are running and reachable.
Request workflow	Postman	Shows repeatable API request construction and response review.
Token analysis	JWT.io, Python decode, Postman	Shows token structure and claim inspection with secrets redacted.
Manual review	Burp Suite	Shows request/response interception and OAuth parameter analysis.
Discovery	Kiterunner, ffuf	Shows endpoint discovery and repeated request behaviour evidence.
GraphQL	DVGA, GraphQL endpoint, curl/Postman	Shows introspection and controlled query testing.

2. API Security Toolchain Overview

A professional API security workflow uses multiple tools because no single tool explains the full attack surface. Docker proves deployment, Postman proves repeatable request construction, JWT.io supports token inspection, Burp shows raw HTTP behaviour, Kiterunner discovers hidden API routes, ffuf measures repeated request behaviour, and GraphQL tooling validates schema and query exposure.

Tool	Role in Methodology	Professional Output
Docker	Deploy and verify lab containers	Container names, status, ports, and uptime.
Browser	Confirm application reachability	Visual confirmation of crAPI or OAuth lab availability.
Postman	Build and replay API requests	Request method, URL, body, headers, response.
JWT.io / local decode	Decode JWT header and payload	Algorithm, claim names, role/user identifiers, expiry metadata.
Burp Suite	Intercept and inspect raw HTTP	Headers, parameters, cookies, OAuth request structure.
Kiterunner	Discover API-aware routes	Endpoint paths, methods, status codes.
ffuf	Repeated request and endpoint behaviour	Status code patterns, response size, word/line count.
DVGA / GraphQL	GraphQL lab testing	Endpoint availability, introspection output, query response.

3. Evidence and Output Matrix

The table below describes the tool-output artifacts that should be retained for professional reporting. Raw secrets must not be uploaded. Every screenshot or output file should be reviewed and redacted before GitHub publication.

Test Area	Tool Output	Reporting Meaning
crAPI lab deployment	docker ps output	Confirms local API lab is running.
Homepage availability	Browser/curl response	Confirms target accessibility.
Postman login + JWT	Login request and token response	Captures auth workflow; token redacted.
JWT claim review	Decoded header/payload	Shows token metadata and claims.
BOLA / IDOR test	Modified object identifier response	Validates authorization behaviour.
Rate-limit behaviour	ffuf output	Shows repeated request behaviour.
Mass assignment	POST body with extra fields	Documents rejected attack/control validation.
Hidden endpoints	Kiterunner route output	Maps undocumented API routes.
GraphQL introspection	Schema discovery response	Shows schema exposure in lab.
OAuth request analysis	Burp request capture	Shows redirect_uri, scope, state, response_type review.

4. Docker and Lab Availability Tool Output

Docker and browser validation are the foundation of the API assessment. They prove that the target is a controlled lab target and that the API service is reachable before any security testing begins.

Tools used

Docker, browser, curl

Representative command / workflow

```
docker ps
curl -I http://127.0.0.1:8888
```

Sanitized output style

NAMES	STATUS	PORTS
crapi-web	Up	0.0.0.0:8888->8888/tcp


```
HTTP/1.1 200 OK
Content-Type: text/html
```

Professional interpretation

The lab is running and reachable. This confirms testing scope and prevents confusion with an unintended target.

Risk and remediation notes

Maintain a lab inventory, use local-only targets where possible, and shut down containers after testing.

5. Postman API Workflow and Authentication Output

Postman provides a structured way to build, replay, and document API requests. In this lab it was used for login, JWT capture, request collections, authorization testing, data exposure review, mass assignment testing, and GraphQL/OAuth workflows.

Tools used

Postman

Representative command / workflow

```
POST http://127.0.0.1:8888/identity/api/auth/login
Content-Type: application/json

{
  "email": "<REDACTED_EMAIL>",
  "password": "<REDACTED_PASSWORD>"
}
```

Sanitized output style

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "token": "<REDACTED_JWT>",
  "type": "Bearer"
}
```

Professional interpretation

The output proves that authentication was captured and can be replayed in a controlled workflow. The token itself is not safe to publish.

Risk and remediation notes

Use environment variables, do not export Postman environments with live tokens, and replace Authorization headers with placeholders before committing.

6. JWT Tooling and Token Analysis Output

JWT analysis was performed to understand token structure, not to publish secrets. Decoding helps identify algorithm, token type, subject, role, expiry, and other claims. A secure API must still verify signatures server-side.

Tools used

JWT.io, local Python decode, Postman

Representative command / workflow

```
python3 - <<PY
import jwt, json
token = "<REDACTED_JWT>"
print(json.dumps(jwt.decode(token, options={"verify_signature": False}), indent=2))
PY
```

Sanitized output style

```
{
  "sub": "<REDACTED_USER>",
  "role": "ROLE_USER",
  "iat": "<REDACTED>",
  "exp": "<REDACTED>"
}
```

Professional interpretation

The output proves that token claims were reviewed. A rejected token tampering attempt should be documented as control validation, not as a vulnerability.

Risk and remediation notes

Reject alg:none, validate issuer/audience/expiry, rotate signing keys when needed, and avoid placing sensitive data in JWT payloads.

7. Burp Suite Proxy and Repeater Output

Burp Suite was used to inspect raw HTTP requests, parameters, headers, cookies, and OAuth request flows. It is important because API bugs often depend on subtle request details that are not obvious in a browser UI.

Tools used

Burp Suite Proxy, Burp Repeater

Representative command / workflow

```
Intercept request -> Send to Repeater -> Modify controlled parameter -> Review response  
body/status/headers
```

Sanitized output style

```
GET /workshop/api/mechanic/mechanic_report?report_id=<TEST_ID> HTTP/1.1  
Host: 127.0.0.1:8888  
Authorization: Bearer <REDACTED_JWT>
```

Professional interpretation

Burp output helps confirm whether behaviour is caused by the server response rather than frontend rendering.

Risk and remediation notes

Redact Cookie, Authorization, access_token, refresh_token, client secrets, OAuth code values, and any personal account identifiers.

8. Kiterunner and ffuf Output Interpretation

Kiterunner and ffuf were used for API discovery and repeated request behaviour testing. Kiterunner is API-aware and useful for route discovery, while ffuf is useful for endpoint fuzzing and volume/behaviour checks.

Tools used

Kiterunner, ffuf

Representative command / workflow

```
kr scan http://127.0.0.1:8888  
ffuf -u http://127.0.0.1:8888/FUZZ -w api-endpoints.txt -t 20
```

Sanitized output style

```
[200] GET /identity/api/auth/login  
[401] GET /workshop/api/mechanic/mechanic_report  
ffuf: status/size/word-count patterns recorded
```

Professional interpretation

These outputs show endpoint discovery and request behaviour. They should be interpreted carefully: repeated requests alone do not prove missing rate limiting unless sufficient evidence shows no throttling over meaningful volume.

Risk and remediation notes

Maintain API inventory, deprecate unused routes, add throttling, return 429 for abuse cases, and monitor repeated failed requests.

9. GraphQL Tool Output Review

GraphQL testing included validating endpoint availability, checking introspection, and querying discovered user-data fields. GraphQL security depends heavily on resolver-level authorization and query restrictions.

Tools used

DVGA, GraphQL endpoint, curl/Postman

Representative command / workflow

```
POST /graphql
Content-Type: application/json

{ "query": "{ __schema { types { name } } }" }
```

Sanitized output style

```
{
  "data": {
    "__schema": {
      "types": [ { "name": "User" } ]
    }
  }
}
```

Professional interpretation

GraphQL introspection and data queries help map the schema and test whether sensitive fields are exposed.

Risk and remediation notes

Disable introspection in production unless required, apply resolver-level authorization, limit query depth/complexity, and return only necessary fields.

10. OAuth Request Analysis Output

OAuth request analysis focused on understanding authorization parameters such as `client_id`, `redirect_uri`, `response_type`, `scope`, `state`, authorization code, and callback URL.

Tools used

Burp Suite, browser, OAuth lab

Representative command / workflow

```
GET /auth?client_id=<CLIENT_ID>&redirect_uri=<CALLBACK>&response_type=code&scope=<SCOPE>&state=<STATE>
```

Sanitized output style

```
OAuth parameters observed:  
client_id=<REDACTED>  
redirect_uri=<REDACTED_CALLBACK>  
response_type=code  
scope=<REDACTED_SCOPE>  
state=<REDACTED_STATE>
```

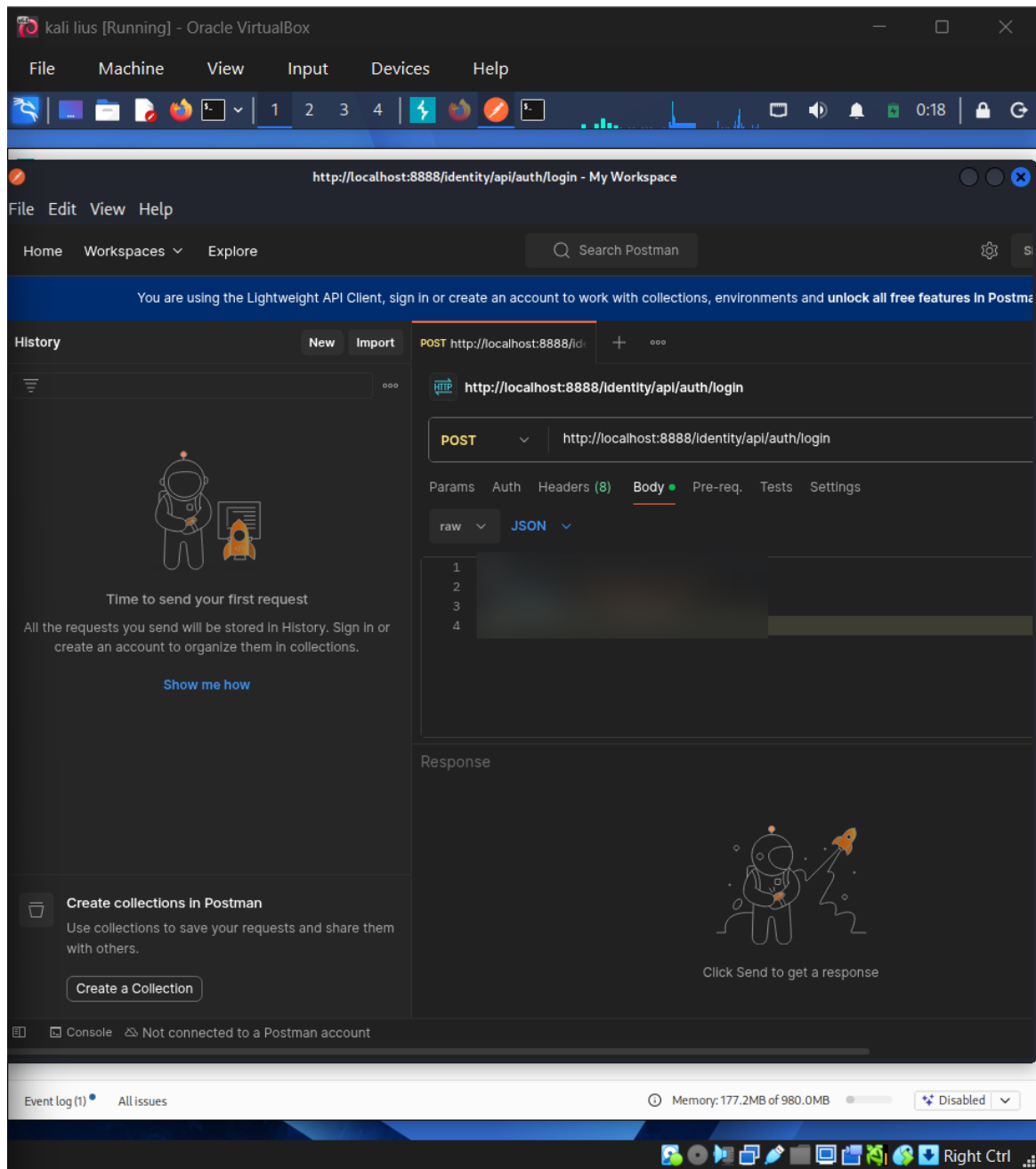
Professional interpretation

The output proves that OAuth request structure was reviewed. The focus is on detecting weak redirect validation, missing state, excessive scope, or leaked authorization codes.

Risk and remediation notes

Strictly validate `redirect_uri`, require unpredictable state, limit scopes, use PKCE where appropriate, and never expose client secrets in frontend code.

Figure A - Postman Authentication Request Evidence



Redacted Postman login request demonstrating API authentication workflow capture.

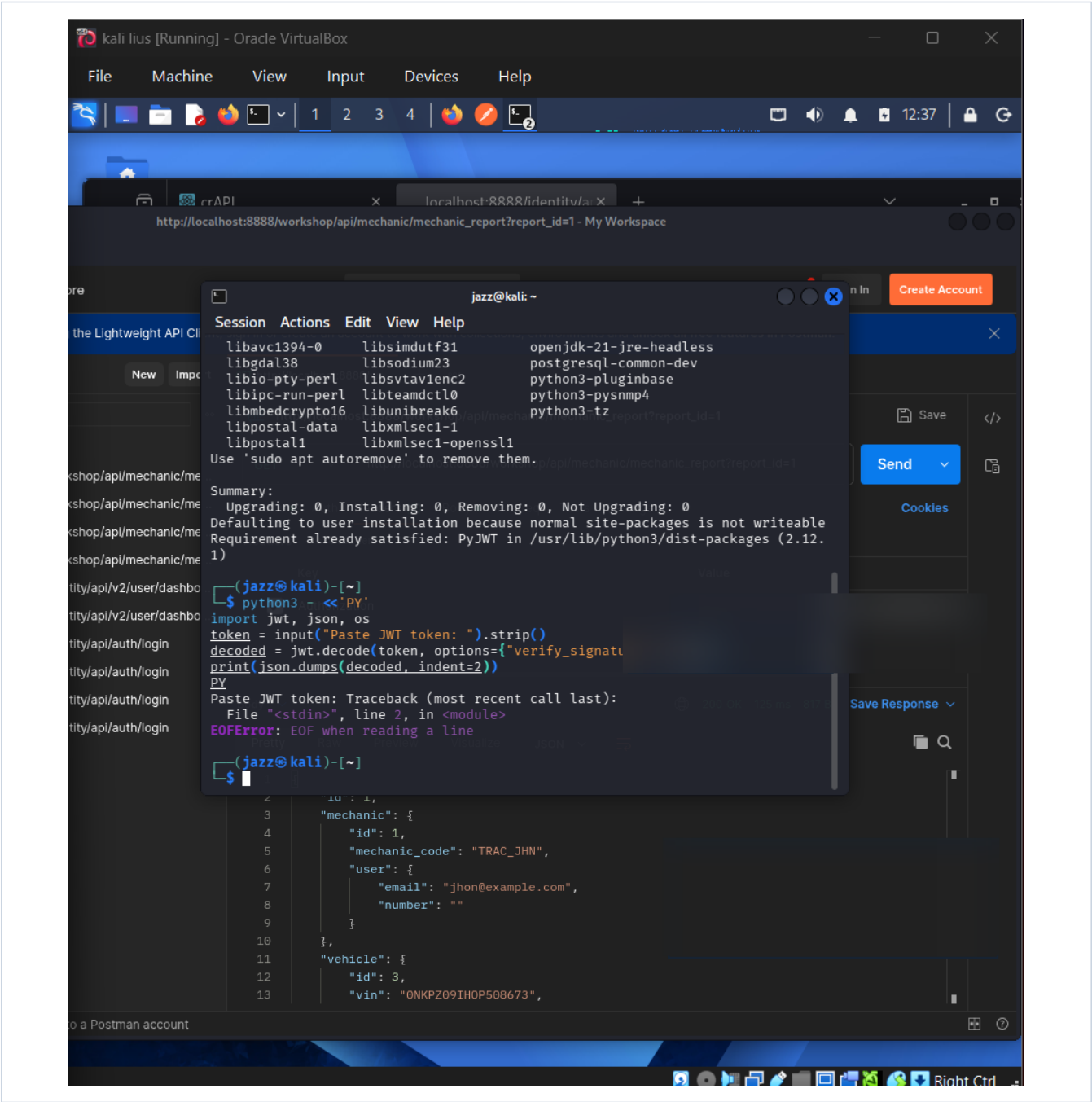
What this screenshot proves

This screenshot proves that the API login request was constructed in Postman and that authentication testing was performed through a repeatable request workflow. The email and password fields are blurred/redacted before publication.

Redaction and alignment note

Sensitive values are redacted or blurred where applicable. The image is centered in a fixed-width evidence frame so that GitHub reviewers can read the screenshot without uneven scaling or layout drift.

Figure B - JWT Decode and Claim Review Evidence



Redacted JWT decode / claim analysis evidence.

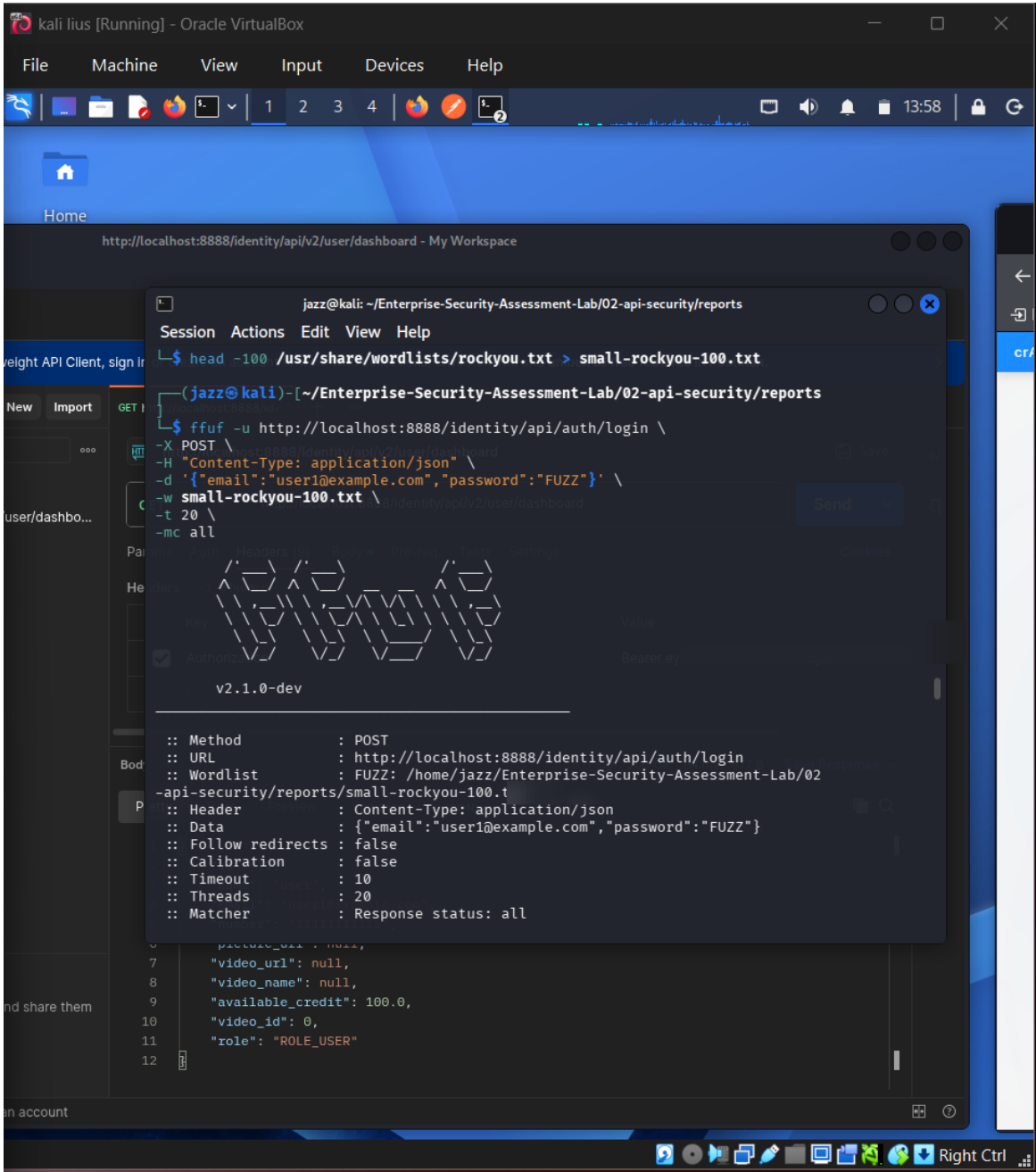
What this screenshot proves

This screenshot proves that JWT token structure and claims were reviewed. Token values and sensitive response fields are redacted to keep the GitHub version safe.

Redaction and alignment note

Sensitive values are redacted or blurred where applicable. The image is centered in a fixed-width evidence frame so that GitHub reviewers can read the screenshot without uneven scaling or layout drift.

Figure C - ffuf Rate-Limit Behaviour Testing Evidence



ffuf repeated request behaviour testing evidence.

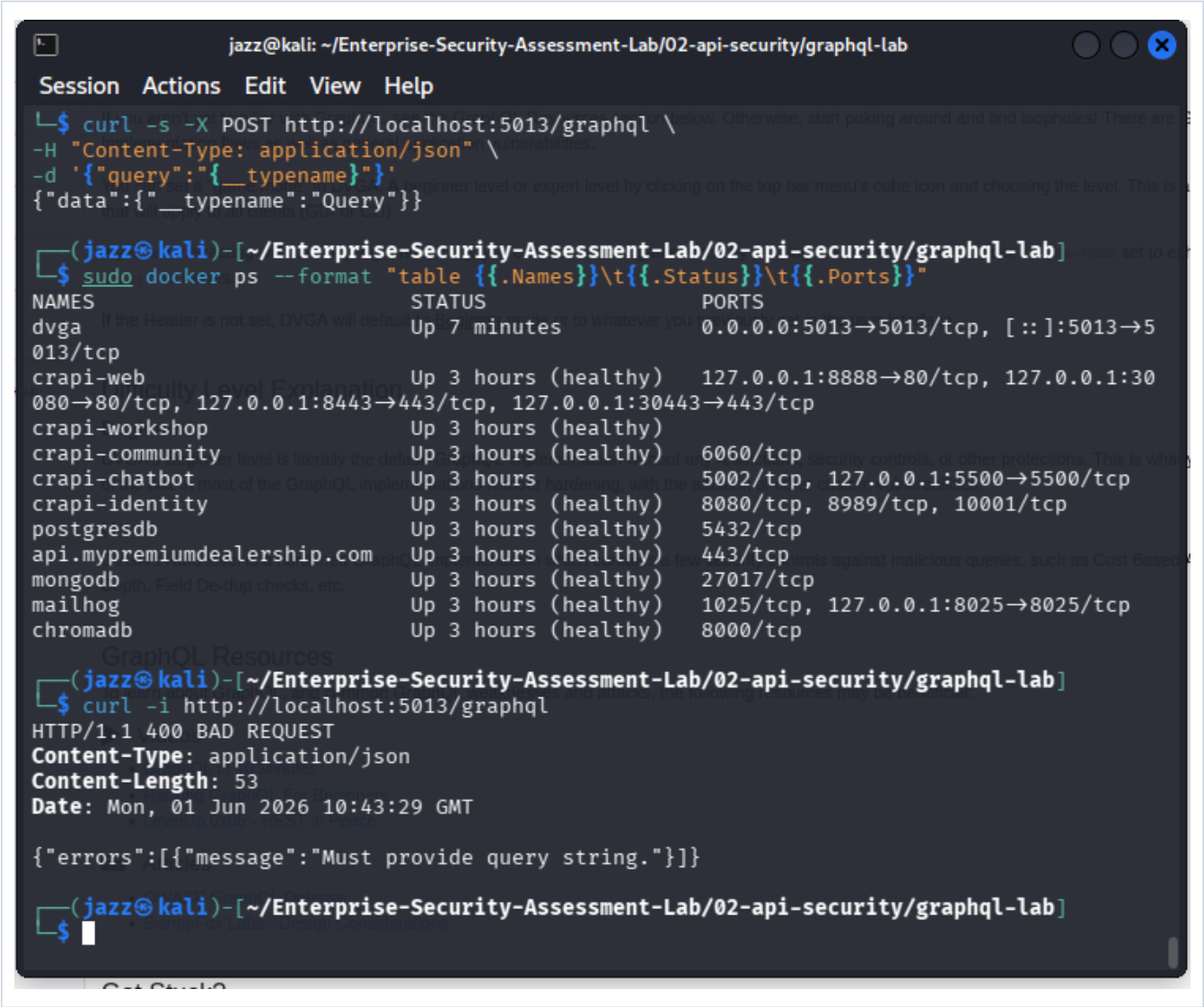
What this screenshot proves

This screenshot proves that ffuf was used to send repeated requests against the authorized lab API. The output supports rate-limit behaviour testing, not destructive brute forcing.

Redaction and alignment note

Sensitive values are redacted or blurred where applicable. The image is centered in a fixed-width evidence frame so that GitHub reviewers can read the screenshot without uneven scaling or layout drift.

Figure D - API Endpoint Discovery Evidence



Endpoint discovery / API mapping evidence.

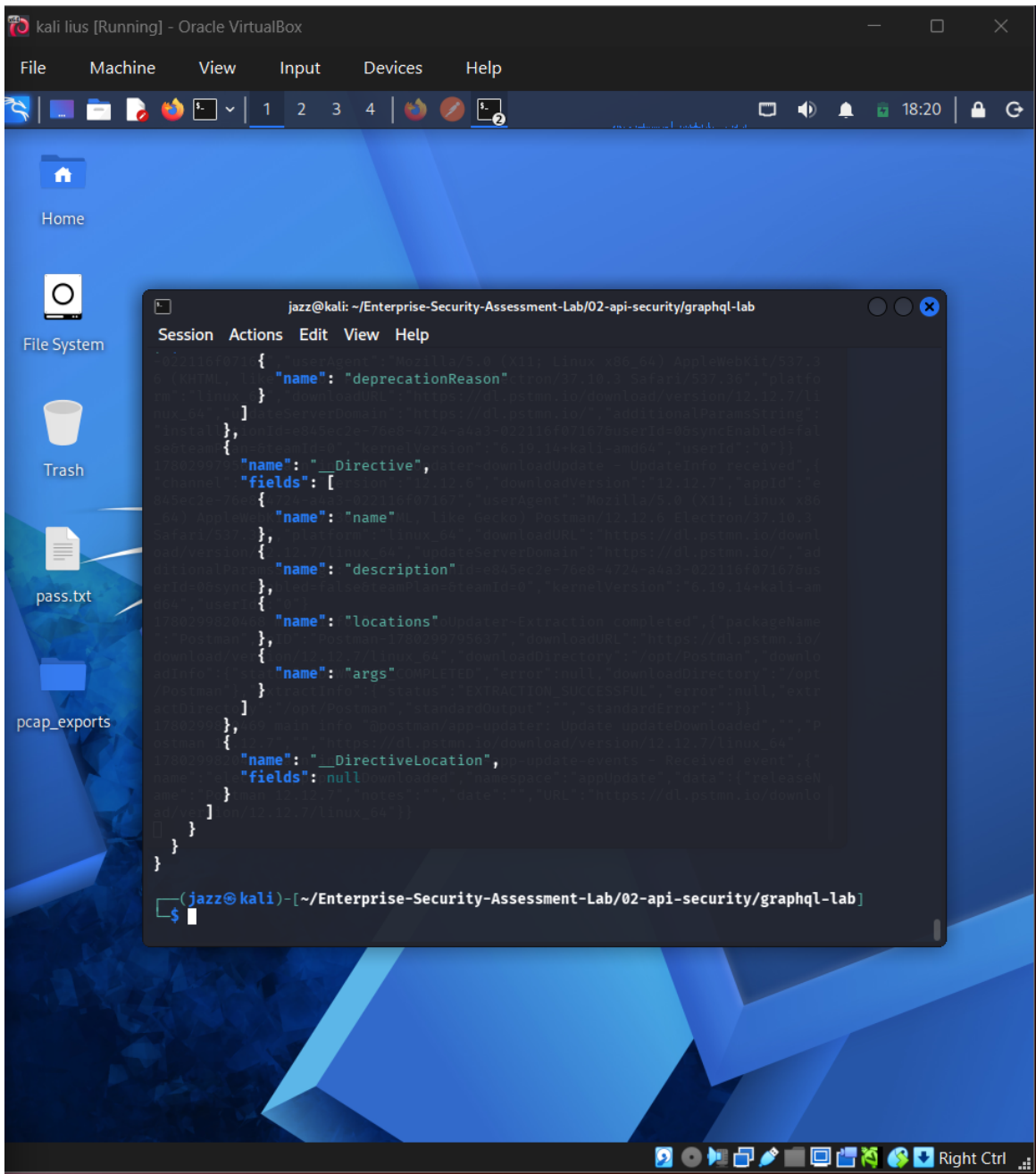
What this screenshot proves

This screenshot proves that API discovery work was performed to identify routes and map the attack surface before deeper testing.

Redaction and alignment note

Sensitive values are redacted or blurred where applicable. The image is centered in a fixed-width evidence frame so that GitHub reviewers can read the screenshot without uneven scaling or layout drift.

Figure E - GraphQL Lab / Endpoint Evidence



GraphQL lab or endpoint availability evidence.

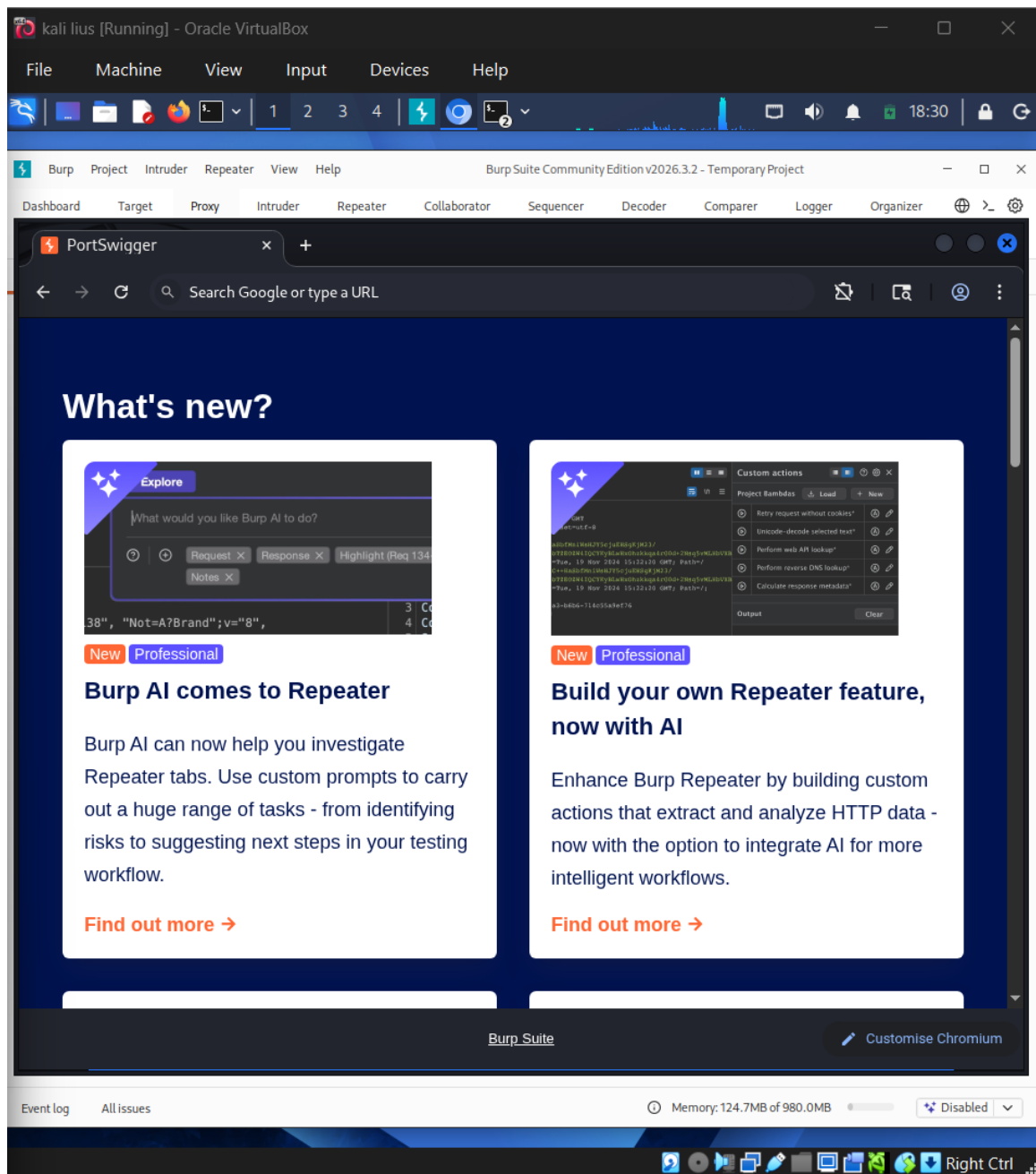
What this screenshot proves

This screenshot proves that the GraphQL test environment was available for controlled schema and query testing.

Redaction and alignment note

Sensitive values are redacted or blurred where applicable. The image is centered in a fixed-width evidence frame so that GitHub reviewers can read the screenshot without uneven scaling or layout drift.

Figure F - GraphQL Introspection Evidence



GraphQL schema discovery / introspection evidence.

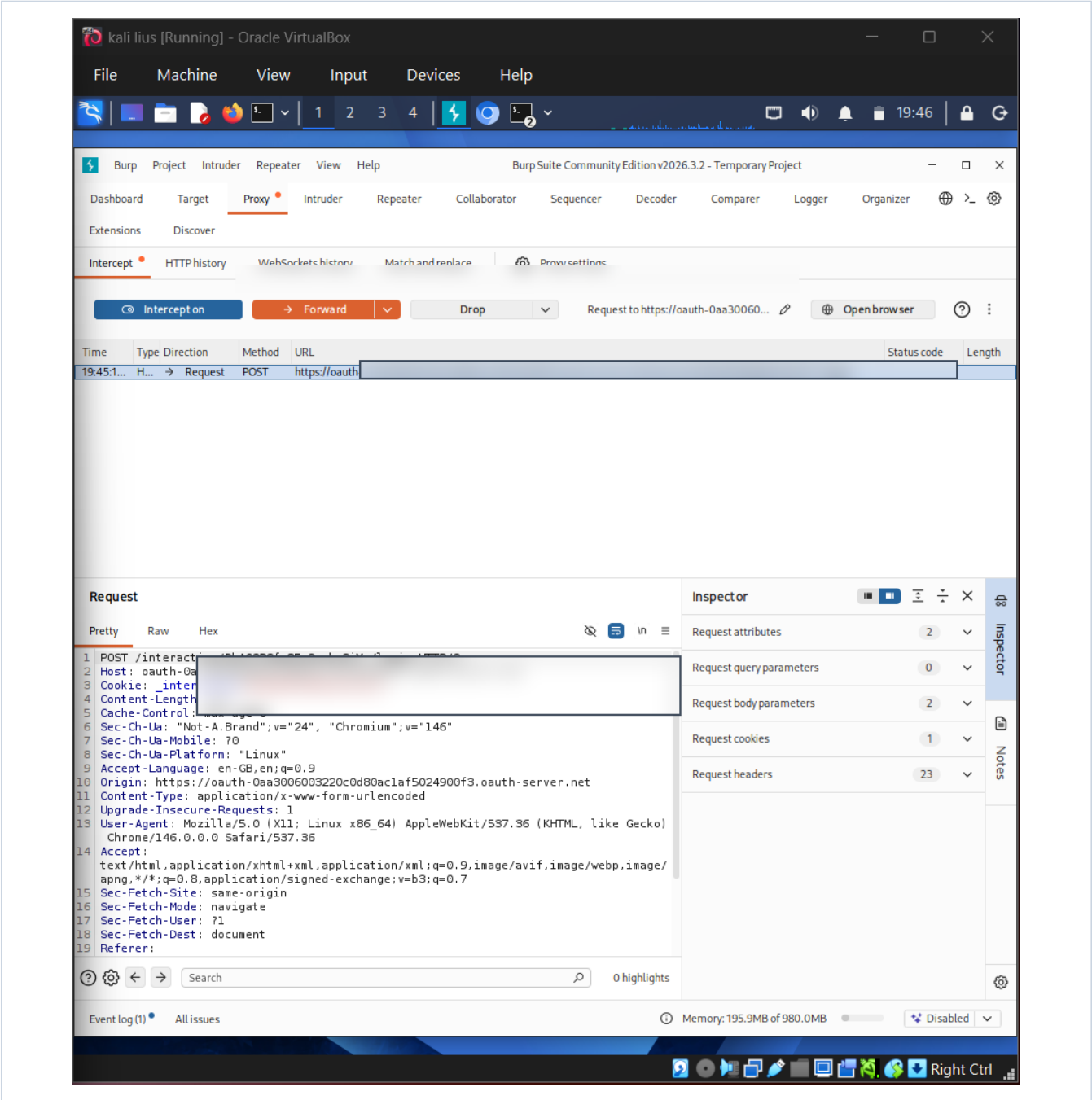
What this screenshot proves

This screenshot proves that GraphQL schema discovery testing was performed. In production, introspection should be controlled based on operational need.

Redaction and alignment note

Sensitive values are redacted or blurred where applicable. The image is centered in a fixed-width evidence frame so that GitHub reviewers can read the screenshot without uneven scaling or layout drift.

Figure G - OAuth Request Analysis Evidence



Redacted OAuth request analysis evidence from Burp/browser.

What this screenshot proves

This screenshot proves that OAuth request parameters were reviewed in a lab context. URL and token-like details are blurred while preserving the testing context.

Redaction and alignment note

Sensitive values are redacted or blurred where applicable. The image is centered in a fixed-width evidence frame so that GitHub reviewers can read the screenshot without uneven scaling or layout drift.

11. Sanitized Output Library

The following examples show how raw tool outputs should be rewritten before being published. The goal is to preserve learning value without exposing tokens, passwords, cookies, JWTs, account data, or private request identifiers.

Docker output

```
docker ps
```

NAMES	STATUS	PORTS
crapi-web	Up	0.0.0.0:8888->8888/tcp

Postman login response

```
{
  "token": "<REDACTED_JWT>",
  "type": "Bearer"
}
```

Burp authorization header

```
Authorization: Bearer <REDACTED_TOKEN>
Cookie: <REDACTED_COOKIE>
```

GraphQL introspection sample

```
{
  "query": "{ __schema { types { name } } }"
```

12. GitHub Redaction and Reporting Guidance

Before uploading API Security evidence, review every screenshot and output file. API evidence often contains tokens in more places than expected: request headers, response bodies, Postman environments, browser URL fragments, OAuth redirects, cookies, and JSON response fields.

Sensitive Item	Action Before GitHub
JWT / Bearer token	Replace with <REDACTED_JWT> or blur the full value.
Password / test password	Blur or replace with <REDACTED_PASSWORD>.
OAuth code / access token / refresh token	Blur the complete value, including URL fragments.
Cookies / session values	Blur Cookie headers and Set-Cookie values.
Email addresses	Use lab placeholders or blur when not required.
Client secret	Never publish; rotate if exposed.
Postman environment files	Do not upload if they contain secrets.
Raw API dumps	Sanitize before committing.

13. Recommended GitHub Structure and README Link

Recommended report path after upload:

02-api-security/reports/api-security-tools-and-tool-output-report-revised.pdf

README link for 02-api-security/README.md:

[ API Security Tools and Tool Output Report](./reports/api-security-tools-and-tool-output-report-revised.pdf)